Course Contents

Unit I

Running with Scissors: Gauging the Threat, Security Concepts, Development Platforms, **Strings**: Character Strings, Common String Manipulation Errors, String Vulnerabilities and Exploits, Mitigation Strategies for Strings, String- Handling Functions.

Unit II

Pointer Subterfuge: Data Locations, Function Pointers, Object Pointers, Modifying the Instruction Pointer, Global Offset Table, The .dtors Section, Virtual Pointers, The atexit() and on_exit() Functions, The longjmp() Function, Exception Handling. **Dynamic Memory Management**: C Memory Management, Common C Memory Management Errors, C++ Dynamic Memory Management, Common C++ Memory Management Errors.

Unit III

Memory Managers, Doug Lea's Memory Allocator, Buffer Overflows on the Heap, Notable Vulnerabilities. **Integer Security**: Introduction to Integer Security, Integer Data Types, Integer Conversions, Integer Operations.

Unit IV

Integer Vulnerabilities, Mitigation Strategies. **Formatted Output**: Variadic Functions, Formatted Output Functions, Exploiting Formatted Output Functions, Stack Randomization, Notable Vulnerabilities.

Unit V

File I/O: File I/O Basics, File I/O Interfaces, Access Control, File Identification, Race Conditions, **Recommended Practices**: The Security Development Lifecycle, Security Training.

Text Book:

1. Robert C. Seacord, Secure Coding in C and C++, 2nd Edition, Pearson, 2013. (Chapter 1,2,3,4,5,6,8,9)

Reference Books:

1. SEI CERT Coding Standards--https://resources.sei.cmu.edu/downloads/secure-coding/assets/sei-cert-c-coding-standard-2016-v01.pdf

Course Objectives:

Introduce Secure Coding Principles
To familiarize students with fundamental security concepts and the importance of writing secure code in C and C++ applications.

Identify and Mitigate String Vulnerabilities
To understand common string manipulation errors, exploits, and develop strategies for secure handling of character strings.

Understand Pointer and Memory Exploits
To explore pointer-related attacks, memory corruption issues, and secure practices in C/C++ memory management.

Enhance Awareness of Integer and Buffer Vulnerabilities
To analyze integer overflows, buffer overflows, and heap vulnerabilities, and apply methods to prevent such attacks.

Secure File I/O Operations
To study secure file handling techniques, mitigate race conditions, and ensure robust access control in file operations.

Print Name Safely:

Write a C program to take your name as input(max 15 characters) and print it:

```
# include<stdio.h>

int main()

{

char name[20];

print(“Enter your name”);
gets(name);  // no bounds check

print(“Hello, %s\n”, name);

return 0;

}
```

```
# include<stdio.h>

int main()

{

char name[20];

print(“Enter your name”);
If(fgets(name, sizeof(name), stdin)!= NULL)
{

name[strcspn(name, “\n”)]=‘\0’;

printf(“Hello, %s\n”, name);

}

else

{
printf(“Input error!\n”);
}
return 0;
}
```

Chapter 1: Running With Scissors

Mrs. Shubha Malige
Asst. Professor
Department of CSE (AI&ML), CSE (Cyber Security)
MSRIT, Bangalore

AGENDA

- **Running with Scissors**
- **Gauging the Threat**
- **Security Concepts**
- **Development Platforms**

RUNNING WITH SCISSORS

- The W32.Blaster.Worm
  - Discovered on August 11, 2003.
  - Infected unpatched system connected to the Internet without user involvement.
  - At least eight million Windows systems have been infected by this worm [Lemos 04].
  - Economic Damage > $500M$

THE W.32 BLASTER WORM

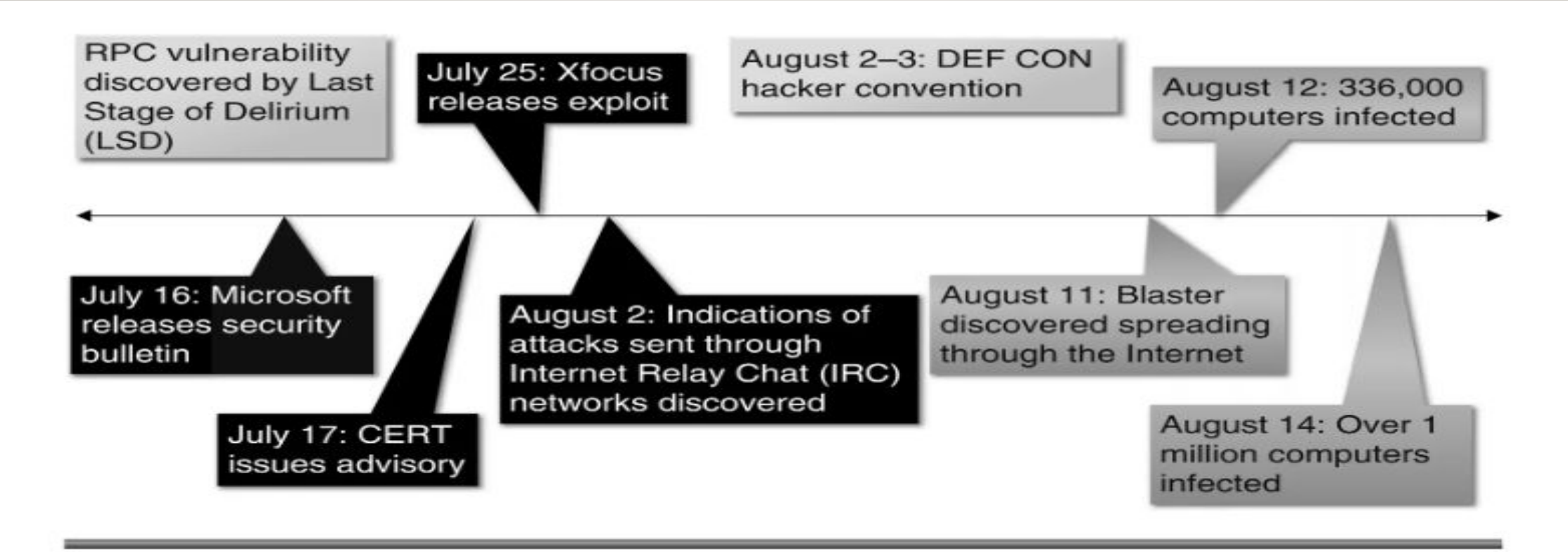


Figure 1.1 Blaster timeline

THE W.32 BLASTER WORM

- Blaster:
  - Checks to see if the computer is already infected.
    - Adds "windows auto update" = "msblast.exe" to registry key HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
  - Runs when Windows starts.
  - Generates a random IP address.
  - Attempts to infect the computer with that address.
  - Sends data on TCP port 135 to exploit the DCOM RPC vulnerability on either Windows XP or Windows 2000.

GAUGING THE THREAT

- The risk of producing insecure software systems can be evaluated by
  - Looking at historic risk.
  - The potential for future attacks.
- Historic risk can be measured by
  - Looking at the type.
  - Cost of perpetrated crimes.
- The potential for future attacks can be partially gauged by:
  - Evaluating emerging threats.
  - The security of existing software systems.

WHAT IS THE COST?

- Based on conservative projections, about 100,000 new software vulnerabilities will be identified in 2010 alone.
- The number of security incidents worldwide will swell to about 400,000 a year, or 8,000 per work week [Berinato 04].

WHAT IS THE COST?

Type of Cybercrime	Global Estimate ($ million)	Reference Period
Cost of Genuine Cybercrime		
Online banking fraud	320	2007
Phishing	70	2010
Malware (consumer)	300	2010
Malware (businesses)	1,000	2010
Bank technology countermeasures	97	2008–10
Fake antivirus	22	2010
Copyright-infringing software	150	2011
Copyright-infringing music, etc.	288	2010
Patent-infringing pharmaceutical	10	2011
Stranded traveler scam	200	2011
Fake escrow scam	1,000[a]	2011
Advance-fee fraud		2011
Cost of Transitional Cybercrime		
Online payment card fraud	4,200[a]	2010
Offline payment card fraud		
Domestic	2,100[a]	2010
International	2,940[a]	2010
Bank/merchant defense costs	2,400	2010

WHO IS THE THREAT?

- Threat is a person, group, organization, or foreign power that has been the source of past attacks or may be the source of future attacks.
- Threats include:
  - Hackers
  - Insiders
  - Criminals
  - Competitive Intelligence Professionals
  - Terrorists
  - Information Warriors.

HACKERS

- Motivated by curiosity and peer recognition from other hackers.
- Write programs that *expose vulnerabilities* in computer software.
- The methods used to disclose these vulnerabilities varies from a policy of responsible disclosure to a policy of full disclosure.

INSIDERS

- The threat comes from a current or former employee or contractor of an organization.
- Has legitimate access to the information that was compromised.
- Do not need to be technically sophisticated to carry out attacks.
- Technically sophisticated insiders can launch attacks with immediate and widespread impact.
- Technical insiders can cover their tracks.

CRIMINALS

- Common crimes include:
  - Auction fraud,
  - Identity theft.
  - Extortion.
- Phishing
  - Lure victims to fake website to gather account data.
- Pfarming
  - Exploit DNS vulnerabilities to redirect web traffic to malicious site.

CORPORATE SPIES

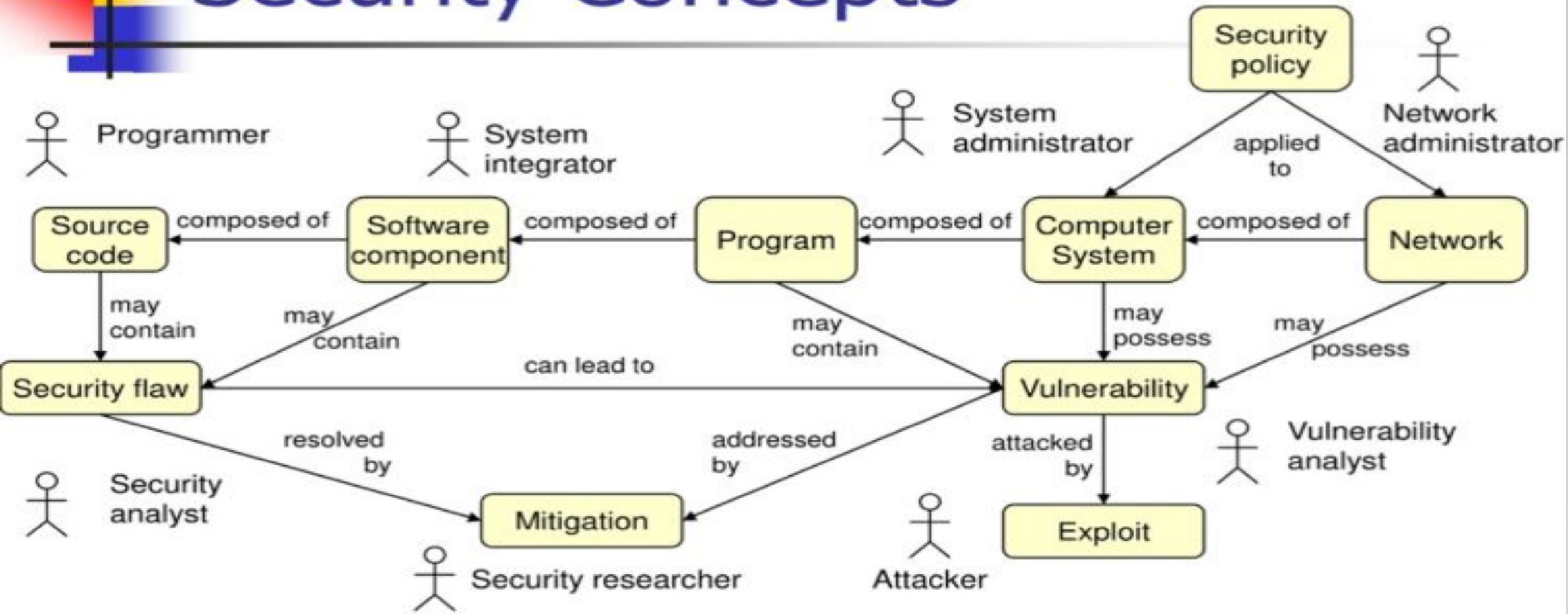
- Corporate spies:
  - Call themselves *competitive intelligence professionals.*
  - Have their own professional association—the Society for Competitive Intelligence Professionals (SCIP).
  - May work from inside a target organization, obtaining employment to steal and market trade secrets.
  - Conduct other forms of corporate espionage.

- Cyber-terrorism can be defined as unlawful attacks or threats of attack against computers, networks, and other information systems to intimidate or coerce a government or its people to further a political or social objective [Denning 00].

INFORMATION WARRIORS

- Eight nations have developed cyber-warfare capabilities comparable to that of the United States.
- More than 100 countries are trying to develop them.
- Twenty-three nations have targeted U.S. systems.

Security Concepts



SECURITY CONCEPTS

- A *programmer* is concerned with properties of source code such as correctness, performance, and security.
- A *system integrator* is responsible for integrating new and existing software components to create programs or systems that satisfy a particular set of customer requirements.
- *System administrators* are responsible for managing and securing one or more systems, including installing and removing software, installing patches, and managing system privileges.
- *Network administrators* are responsible for managing the secure operations of networks.
- A *security analyst* is concerned with properties of security flaws and how to identify them.
- A *vulnerability analyst* is concerned with analyzing vulnerabilities in existing and deployed programs.

SECURITY CONCEPTS

- A *security researcher* develops mitigation strategies and solutions and may be employed in industry, academia, or government.
- The *attacker* is a malicious actor who exploits vulnerabilities to achieve an objective. These objectives vary depending on the threat. The attacker can also be referred to as the adversary, malicious user, hacker, or other alias.

SECURITY CONCEPTS

Security Policy

A security policy is a set of rules and practices that are normally applied by system and network administrators to their systems to secure them from threats. The following definition is taken verbatim from RFC 2828, the *Internet Security Glossary* [Internet Society 2000]:

> **Security Policy**
> A set of rules and practices that specify or regulate how a system or organization provides security services to protect sensitive and critical system resources.

Security policies can be both implicit and explicit. Security policies that are documented, well known, and visibly enforced can help establish expected user behavior. However, the lack of an explicit security policy does not mean an organization is immune to attack because it has no security policy to violate.

SECURITY CONCEPTS

Security Flaws

Software engineering has long been concerned with the elimination of *software defects*. A software defect is the encoding of a human error into the software, including omissions. Software defects can originate at any point in the software development life cycle. For example, a defect in a deployed product can originate from a misstated or misrepresented requirement.

Security Flaw
A software defect that poses a potential security risk.

Not all software defects pose a security risk. Those that do are *security flaws*. If we accept that a security flaw is a software defect, then we must also accept that by eliminating all software defects, we can eliminate all security flaws.

SECURITY CONCEPTS

- *Computer security* is preventing attackers from achieving objectives through unauthorized access or unauthorized use of computers and networks [Howard 97].
- A *programmer* is concerned with properties of source code such as correctness, performance, and security.
- A *system integrator* is responsible for integrating new and existing software components to create programs or systems that satisfy a particular set of customer requirements.
- *System administrators* are responsible for managing and securing one or more systems including installing and removing software, installing patches, and managing system privileges.

SECURITY CONCEPTS

- *Network administrators* are responsible for managing the secure operations of networks.
- A *security analyst* is concerned with properties of security flaws and how to identify them.
- A *vulnerability analyst* is concerned with analyzing vulnerabilities in existing and deployed programs.

SECURITY CONCEPTS

- A *security researcher* develops mitigation strategies and solutions and who may be employed in industry, academia, or government.

- The *attacker:*
  - Is a malicious actor who exploits vulnerabilities to achieve an objective.
  - These objectives vary depending on the threat.
  - The attacker can also be referred to as the adversary, malicious user, hacker, or other alias.

SECURITY POLICY

- A Security Policy is a set of rules and practices that specify or regulate how a system or organization provides security services to protect sensitive and critical system resources

SECURITY FLAWS

- **Software Defects:**
  - A software defect is the encoding of a human error into the software, including omissions.

Security Flaw:

- A *security flaw* is a software defect that poses a potential security risk.
- Eliminating software defects eliminate security flaws.

VULNERABILITIES

- *Vulnerability*
  - set of conditions that allows an attacker to violate an explicit or implicit security policy.
  - Not all security flaws lead to vulnerabilities.
- *Security flaw*
  - can cause a program to be vulnerable to attack.
- Vulnerabilities can also exist without a security flaw.

EXPLOITS

- Exploit:
  - Proof-of-concept exploits are developed to prove the existence of a vulnerability.
  - Proof-of-concept exploits are beneficial when properly managed.
  - Proof-of-concept exploit in the wrong hands can be quickly transformed into a worm or virus or used in an attack.

MITIGATION

- **Mitigation:**
  - *Mitigations* are methods, techniques, processes, tools, or runtime libraries that can prevent or limit exploits against vulnerabilities.
  - At the source code level, a mitigation might be replacing an unbounded string copy operation with a bounded one.
  - At a system or network level, a mitigation might involve turning off a port or filtering traffic to prevent an attacker from accessing a vulnerability.

C AND C++

- C and C++:
  - Popular programming languages.
  - The vast majority of vulnerabilities that have been reported to the CERT/CC have occurred in programs written in one of these two languages.

WHAT IS THE PROBLEM WITH C?

- Short term solutions:
  - Educating developers in how to program securely by recognizing common security flaws and applying appropriate mitigations.
- Long term solutions:
  - Language standard, compilers, and tools evolve.

LEGACY CODE

- A significant amount of legacy C code was created (and passed on) before the standardization of the language.
- Legacy C code is at higher risk for security flaws because of the looser compiler standards and is harder to secure because of the resulting coding style.

OPERATING SYSTEMS

Microsoft Windows:- Microsoft Windows family of operating system products, including Windows 7, Windows Vista, Windows XP, Windows Server 2003, Windows 2000.

Linux: Linux is free Unix Derivative created by Linus Torvalds with the assistance of developers around the world.

OTHER LANGUAGES

- Many security professionals recommend using other languages, such as Java.
- Adopting Java is often not a viable option because of:
  - Existing investment in C source code,
  - Programming expertise,
  - Development environments.
- Another alternative to using C is to use a C dialect, such as Cyclone [Jim 02].
- Cyclone is currently supported on x86 Linux, and on Windows using Cygwin.

COMPILERS

- Microsoft's Visual C++ is the predominant C and C++ compiler on Windows platforms.
- Visual C++ includes
  - Visual C++ 6.0
  - Visual C++ .NET 2002
  - Visual C++ .NET 2003
  - Visual C++ 2005 Beta1 and Beta2
- The GCC compilers are the predominant C and C++ compilers for Linux platforms.

DEVELOPMENT PLATFORMS

Operating Systems

Microsoft Windows. Many of the examples in this book are based on the Microsoft Windows family of operating system products, including Windows 7, Windows Vista, Windows XP, Windows Server 2003, Windows 2000, Windows Me, Windows 98, Windows 95, Windows NT Workstation, Windows NT Server, and other products.

Linux. Linux is a free UNIX derivative created by Linus Torvalds with the assistance of developers around the world. Linux is available for many different platforms but is commonly used on Intel-based machines.

DEVELOPMENT PLATFORMS

Compilers

The choice of compiler and associated runtime has a large influence on the security of a program. The examples in this book are written primarily for Visual C++ on Windows and GCC on Linux, which are described in the following sections.

Visual C++. Microsoft's Visual C++ is the predominant C and C++ compiler on Windows platforms. Visual C++ is actually a family of compiler products that includes Visual Studio 2012, Visual Studio 2010, Visual Studio 2008, Visual Studio 2005, and older versions. These versions are all in widespread use and vary in functionality, including the security features provided. In general, the newer versions of the compiler provide more, and more advanced, security features. Visual Studio 2012, for example, includes improved support for the C++11 standard.

GCC. The GNU Compiler Collection, or GCC, includes front ends for C, C++, Objective-C, Fortran, Java, and Ada, as well as libraries for these languages. The GCC compilers are the predominant C and C++ compilers for Linux platforms.